BOARD GAME CLUB

Final Project Report

Course: Software Design Patterns

Team members: Alisher Kairov, Sanzhar Zhunis

Repository: https://github.com/jostartir/BoardGame-Club

1. INTRODUCTION

Board Game Club helps people organize board game nights for friends, classmates or a small club. The organizer adds the participants, sets the time available for the meeting and picks a game from the recommendations. After playing, the organizer can save scores, correct them if needed and download a report. Saved results are available when the organizer opens the meeting again.

We built the application for the Software Design Patterns course. The game logic is written in Java and uses six patterns: Builder, Facade, Decorator, Adapter, Strategy and Observer. We used HTML, CSS and JavaScript for the browser interface, Maven for the build and JUnit 5 for automated tests.

2. MAIN BODY

2.1. User scenarios

1. Create a meeting. Enter a title, a date and time, and the number of minutes available. Click Create meeting. The application opens the meeting and adds it to Saved meetings.
2. Manage participants. Add or remove players before starting a game. The recommendations update after each change. Chess appears for two players, disappears when a third person joins and comes back if that person is removed.
3. Pick a game. Choose By duration or By difficulty. The application filters games by player count and available time, then sorts the matches using the selected strategy. Select a game and click Start game to create the session and open the score form.
4. Record the result. Save each player's score as the game progresses. Before clicking Finish game, enter a score for every player and a positive number of minutes played. The result shows the scores and the winner or winners. To correct a score after the game ends, use Fix scores. The game stays finished and its recorded duration stays the same.

5. Reopen a meeting and download a report. Open the meeting from Saved meetings after restarting the application. The page shows the participants, game, playing time and saved scores. Choose whether to include playing time and scores in the report, then download meeting-report.txt.

2.2. Architecture and data flow

The browser displays an HTML page styled with CSS. When the organizer clicks a button, JavaScript sends an HTTP request to WebServer. The server serves the page and its assets, passes requests to MeetingController and returns the application state as JSON. The controller calls the classes that create meetings, recommend games, record scores and save data.

BoardGame holds the title, player range, expected duration and difficulty of a game in the catalog. GameSession holds the record of a particular game played at a GameNight. A Participant represents one person in the group, and a PlayerResult stores that person's score for the session.

Main reads games.csv through the catalog adapter, creates the controller and starts the local server. JsonMeetingRepository reads saved meetings from data/meetings.json and restores their Java objects, scores and actual durations. After an action such as starting a game or correcting a score, the controller passes the full meeting list to the repository. Gson converts the records to JSON. The repository writes the JSON to a temporary file before replacing the saved file.

The separation between game rules and file access follows the single responsibility principle. GameSession applies the rules for recording scores and finishing a session, and JsonMeetingRepository handles reading and writing. GameRecommender depends on RecommendationStrategy, which gives both recommendation algorithms the same contract. ParticipantObserver and MeetingReport each expose a single operation. Controller methods such as startGame, saveScore and correctScore identify the action they perform.

To run the project, use JDK 25 and Maven. The README gives instructions for starting it from the project root and opening http://localhost:8080/. It also describes the sample catalog, the saved data and the test command.

2.3. Design pattern table

Pattern	Category	Problem solved	Main classes and interfaces	Demonstration	Responsible team member
Builder	Creational	Collect the settings from the meeting form before creating a GameNight.	GameNightBuilder, GameNight	Complete the form and click Create meeting.	Alisher K.
Facade	Structural	Give the controller one method to check the selected game and start a session.	GameNightFacade, GameNight, GameSession	Select a suitable game and click Start game.	Alisher K.
Decorator	Structural	Let the organizer add playing time, scores or both to the basic report.	MeetingReport, BasicMeetingReport, DurationReportDecorator, ScoresReportDecorator	Toggle Include playing time and Include scores.	Alisher K.
Adapter	Structural	Read the CSV records as BoardGame objects.	GameCatalogSource, CsvReader, CsvRecord, CsvGameCatalogAdapter	Open the catalog and see Uno, Monopoly and Chess from games.csv.	Sanzhar Z.
Strategy	Behavioral	Give the organizer two ways to sort the games that fit the meeting.	RecommendationStrategy, AbstractRecommendationStrategy, ByDurationStrategy,	Switch between By duration and By difficulty.	Sanzhar Z.

			ByDifficultyStrategy, GameRecommender		
Observer	Behavioral	Update recommendations when a participant joins or leaves.	ParticipantService, ParticipantEvent, ParticipantObserver, RecommendationUpdater	Add a third player and see Chess disappear from the recommendations.	Sanzhar Z.

2.4. UML diagrams

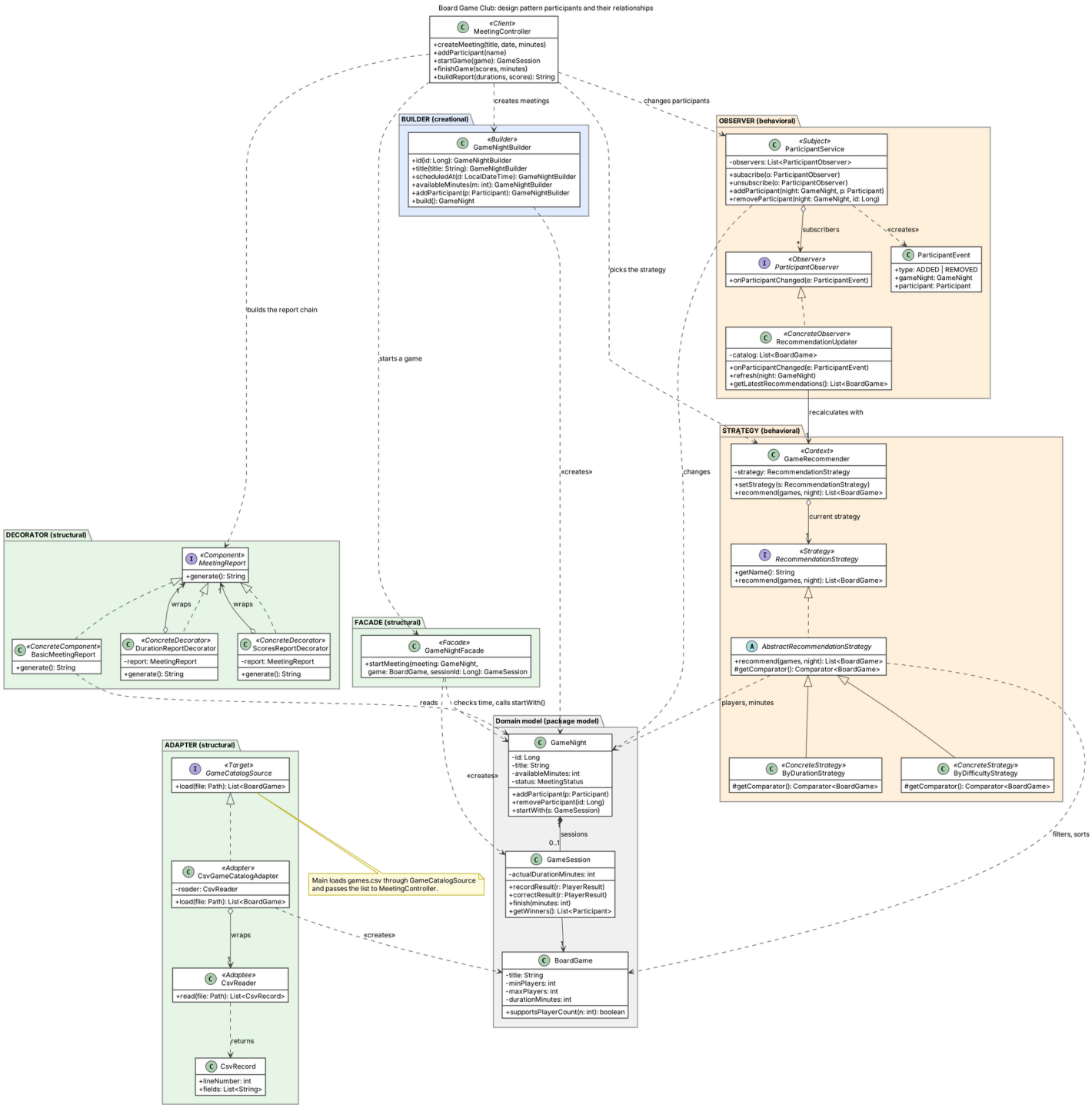


Figure 1. Class diagram showing the domain objects and the classes involved in the six patterns.

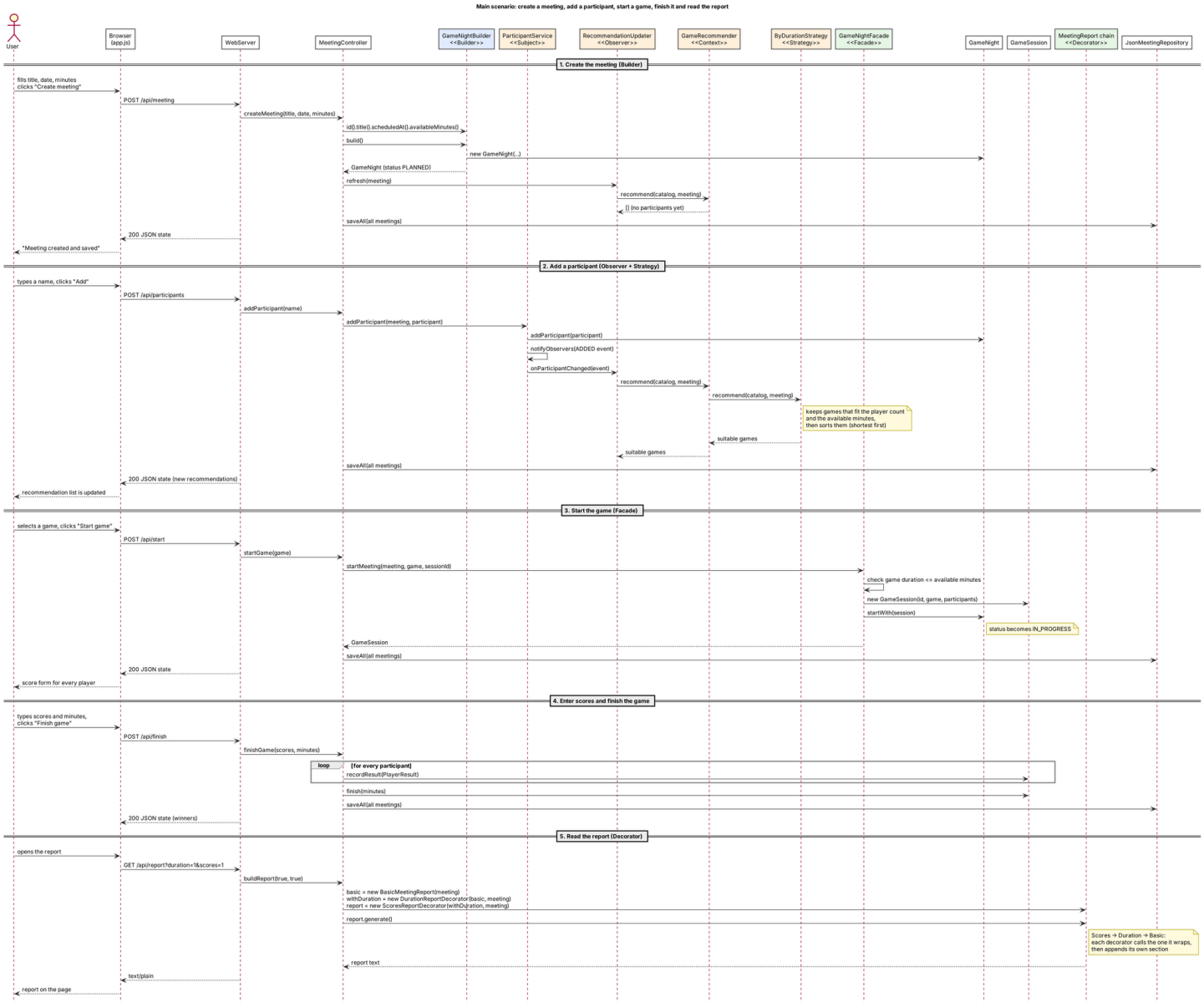


Figure 2. Sequence diagram from creating a meeting and changing its participants to starting a game, finishing it and requesting the report.

2.5. Interface screenshots

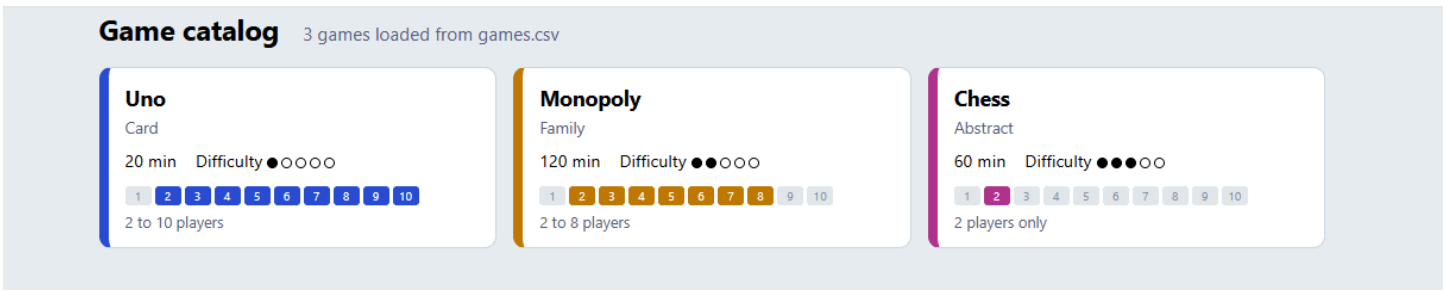
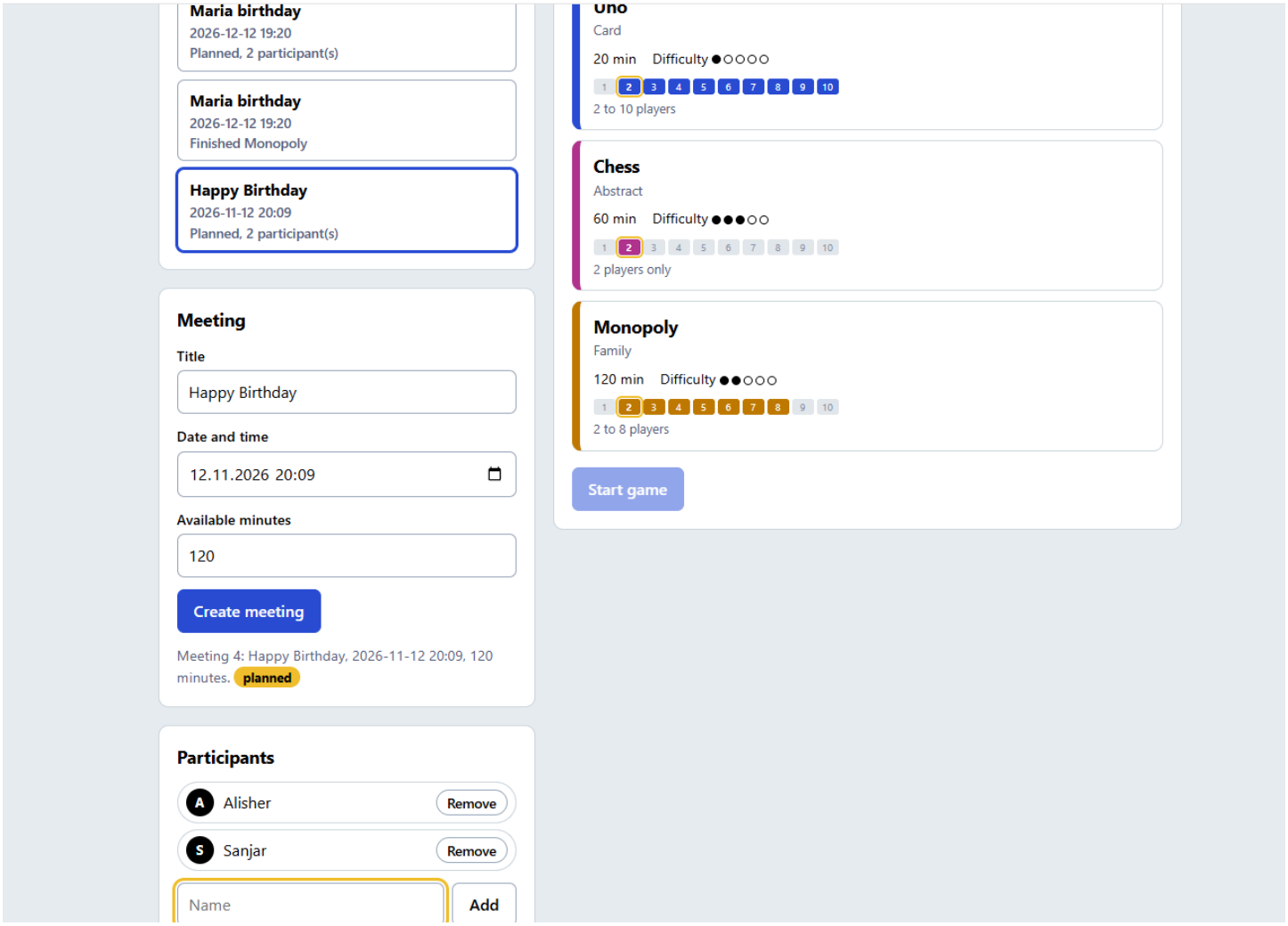


Figure 3. The catalog shows each game's player range, duration and difficulty. The games are loaded from CSV.


Maria birthday
2026-12-12 19:20
Planned, 2 participant(s)
Maria birthday
2026-12-12 19:20
Finished Monopoly
Happy Birthday
2026-11-12 20:09
Planned, 2 participant(s)
Meeting
Title
Happy Birthday
Date and time
12.11.2026 20:09
Available minutes
120
Create meeting
Meeting 4: Happy Birthday, 2026-11-12 20:09, 120 minutes. planned
Participants
A Alisher Remove
S Sanjar Remove
Name
Add
Uno
Card
20 min Difficulty ●○○○○
1 2 3 4 5 6 7 8 9 10
2 to 10 players
Chess
Abstract
60 min Difficulty ●●●○○
1 2 3 4 5 6 7 8 9 10
2 players only
Monopoly
Family
120 min Difficulty ●●○○○
1 2 3 4 5 6 7 8 9 10
2 to 8 players
Start game

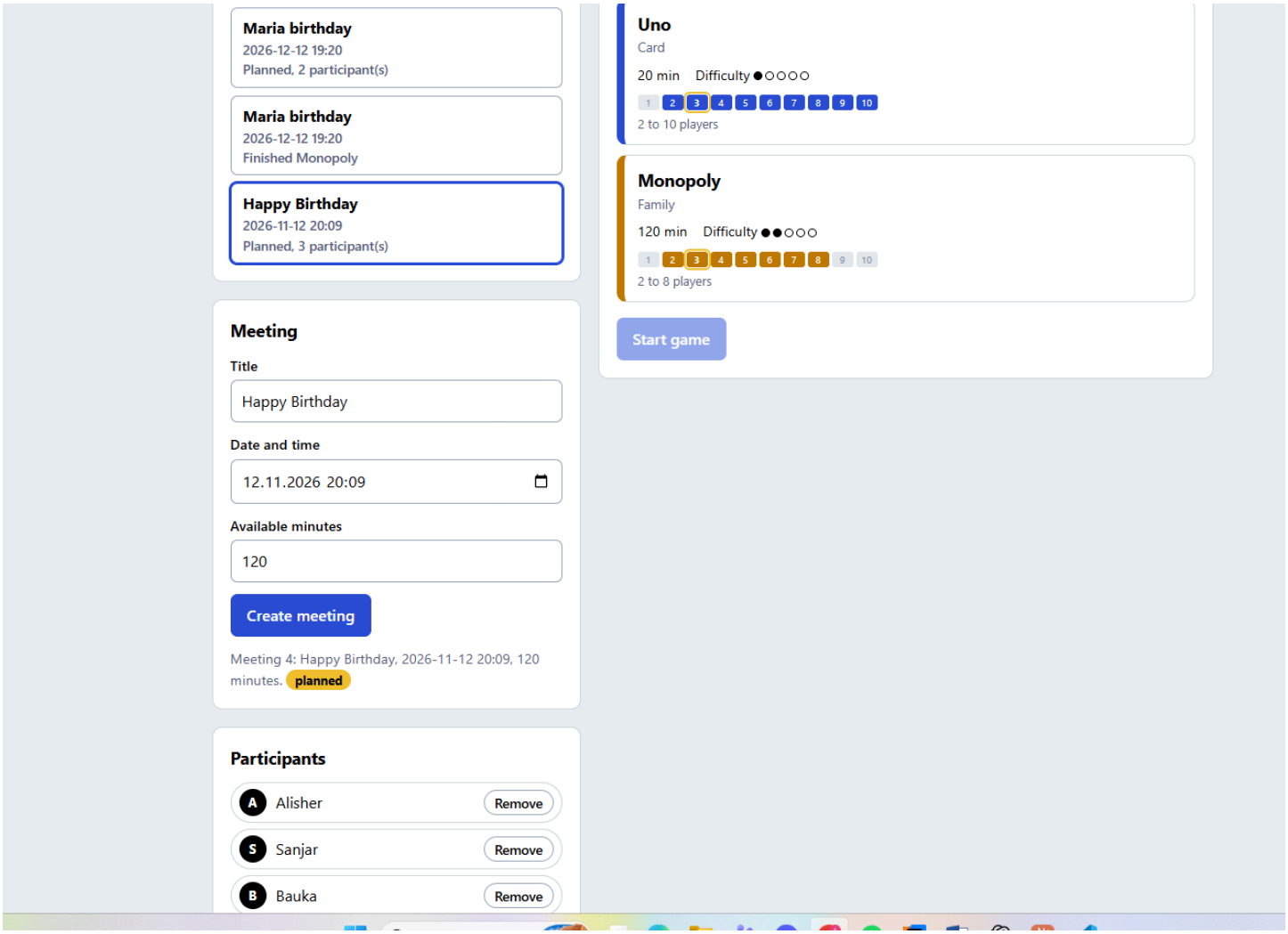


Figure 4. Chess appears for two players. Adding a third player removes it from the recommendations.

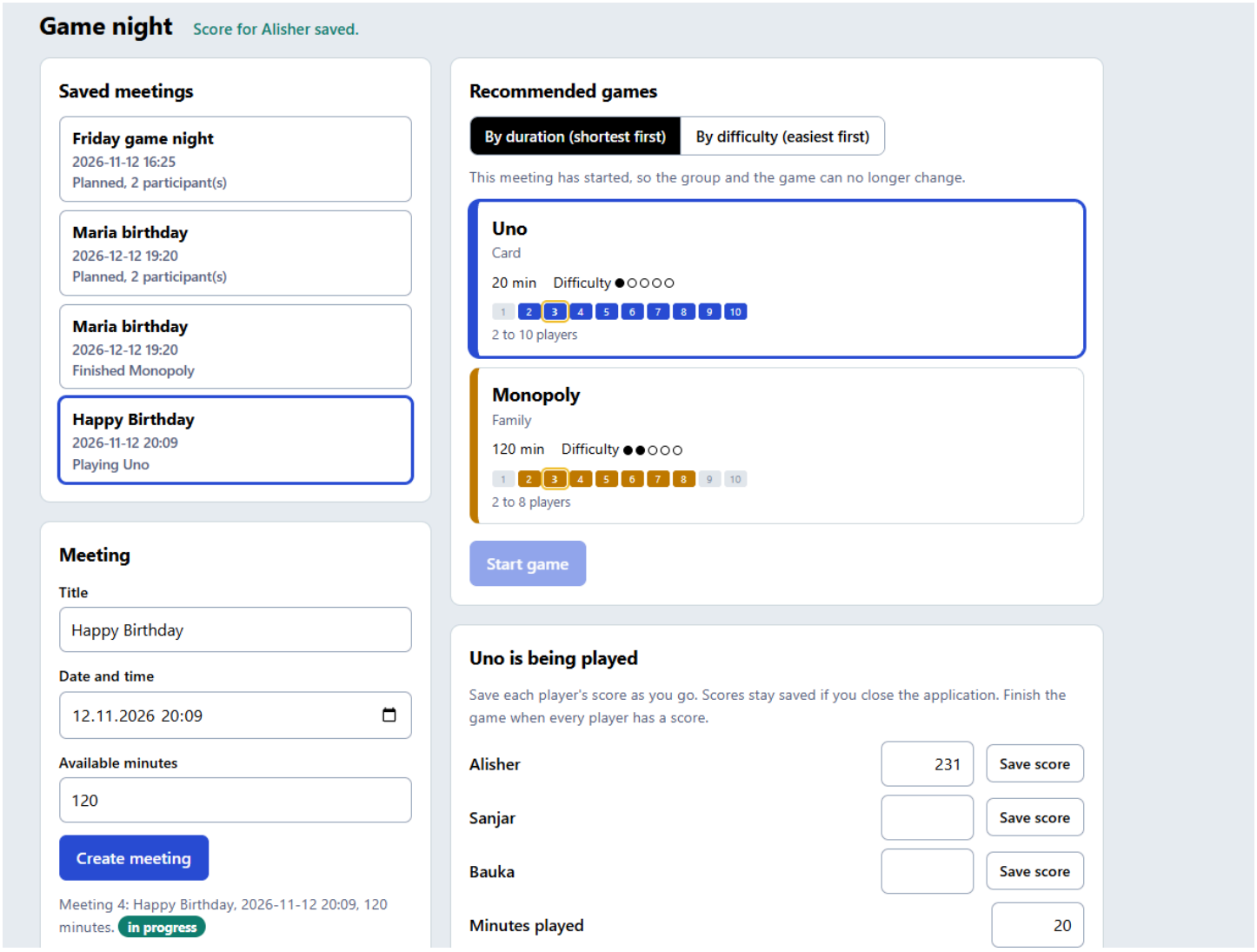


Figure 5. The organizer saves one player's score while the game is still in progress.

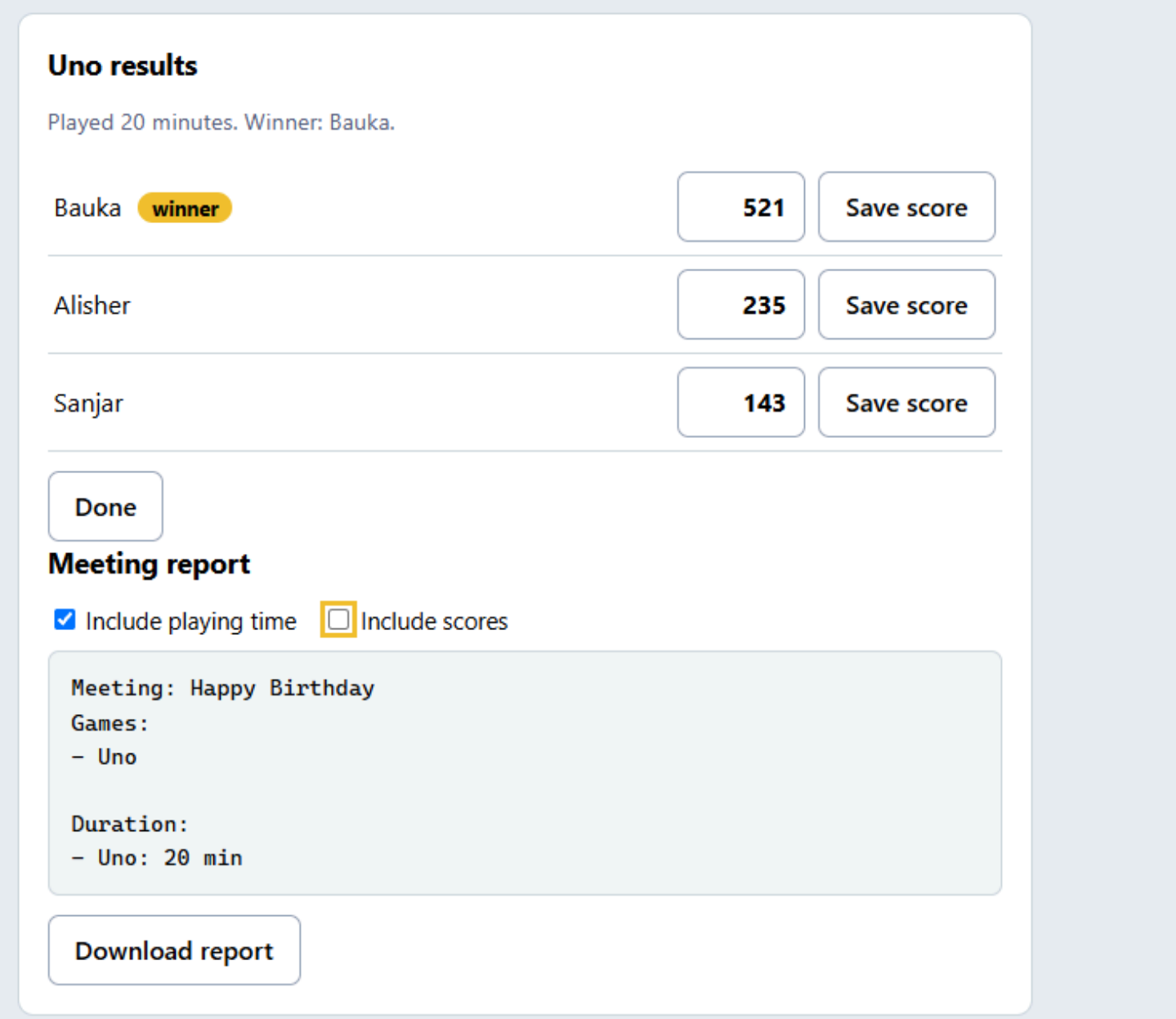


Figure 6. The finished game shows the corrected score. The report checkboxes let the organizer choose whether to download the report with playing time and scores.

2.6. Testing and results

All 57 JUnit tests across eight classes passed. The suite covers the individual patterns, the meeting controller and the HTTP interface. To repeat the test run, use mvn test.

Test class	Tests passed	Coverage
GameNightBuilderTest	3	Meeting settings, default duration and duplicate participant IDs
GameNightFacadeTest	3	Session creation and rejection of unsuitable games
MeetingReportDecoratorTest	2	Combined report sections and corrected scores in the report
CsvGameCatalogAdapterTest	8	CSV loading, quoted titles and invalid records
RecommendationStrategyTest	5	Filtering, both sorting orders and empty recommendations
ParticipantObserverTest	7	Participant notifications and recommendation updates
MeetingControllerTest	21	Meeting workflow, score correction, persistence and error handling
WebServerTest	8	Page and catalog responses, meeting actions, scores and invalid requests
Total	57	All passed

Representative test scenarios

Scenario and input	Expected result	Reported result
Build a meeting with a title, date, 90 minutes and two participants.	The meeting contains the supplied values and both participants.	Passed
Change the strategy from duration to difficulty.	The same suitable games are returned in the order specified by the active strategy.	Passed
Add a third participant to a two-player meeting that recommends Chess.	Chess disappears from the recommendations.	Passed
Start a game lasting exactly 60 minutes in a meeting with 60 minutes available.	A session is created and the meeting moves to IN_PROGRESS.	Passed
Try to start a 61-minute game with only 60 minutes available.	The action is rejected and the meeting remains planned.	Passed
Try to create a meeting using an invalid date through the HTTP interface.	The request is rejected with an error response.	Passed
Try to finish a game with a missing score or zero minutes played.	The application rejects the finish operation.	Passed
Save a score, reload the meeting, correct the finished result and reload again.	The original score and the later correction survive their respective reloads.	Passed
Generate the report with both score and duration sections enabled.	The result includes the basic report, player scores and actual duration.	Passed

2.7. Individual contributions

Alisher Kairov: implemented meeting creation with Builder, session startup with Facade and the optional report sections with Decorator. They also implemented JSON storage for meetings and scores and wrote JUnit tests for those three patterns.

Sanzhar Zhunis: implemented the two recommendation strategies, the observer that updates recommendations when players change and the adapter that loads the CSV catalog. They built the browser interface and connected its controls to the Java application.

We worked together to connect both sets of features and prepare the documentation and UML diagrams.

3. CONCLUSION

An organizer can use Board Game Club for a complete session, from setting the date and choosing a game to correcting the final score and downloading a report. Saving the meeting makes it possible to close the application and return to the same results later.

The six patterns divide the work between meeting creation, catalog import, recommendations, session startup and report generation. MeetingController connects those operations to the browser. The automated tests passed for the separate components and for the actions that use them together.

4.FUTURE WORK

Allow several game sessions in one meeting. The current version records one game per meeting. With more sessions, the organizer could record a whole evening and download one report for it.

Let organizers edit planned meetings and delete saved entries from the interface.

Add a screen for editing the catalog and validate the fields that the CSV importer uses.

Keep JSON for local use and consider a database if several organizers need to save changes at the same time.

5. Java implementation excerpts

Builder — MeetingController.createMeeting

```
GameNight created = new GameNightBuilder()
        .id(nextMeetingId)
        .title(title)
        .scheduledAt(scheduledAt)
        .availableMinutes(availableMinutes)
        .build();
```

The method names show which settings the controller passes from the form. The builder holds those values until build() creates the GameNight. It can also accept participants during construction. If the caller omits the duration, the builder uses 120 minutes.

Facade — GameNightFacade.startMeeting

```
if (game.getDurationMinutes() > meeting.getAvailableMinutes()) {
    throw new IllegalArgumentException(
            "Game takes longer than the meeting allows");
}

GameSession session = new GameSession(
        sessionId, game, meeting.getParticipants());
meeting.startWith(session);
return session;
```

The facade checks that the game fits the available time. The GameSession constructor checks the number of players. GameNight then records the session and updates its status. Once startMeeting() returns, the controller saves the meeting.

Decorator — MeetingController.buildReport

```
MeetingReport report = new BasicMeetingReport(meeting);


if (withDurations) {
    report = new DurationReportDecorator(report, meeting);
}


if (withScores) {
    report = new ScoresReportDecorator(report, meeting);
}


return report.generate();
```

BasicMeetingReport lists the meeting and game. Each decorator implements MeetingReport and calls generate() on its wrapped report before adding its own section. The controller wraps the report according to the two checkboxes, so the organizer can choose either section or both.

Adapter — CsvGameCatalogAdapter.toBoardGame

```
return new BoardGame(
        Long.parseLong(f.get(0)),
        f.get(1),
        Integer.parseInt(f.get(2)),
        Integer.parseInt(f.get(3)),
        Integer.parseInt(f.get(4)),
        Integer.parseInt(f.get(5)),
        f.get(6)
);
```

CsvReader returns a CsvRecord with text fields and a line number. The adapter checks the field count, parses the numeric columns and constructs a BoardGame. If a value is invalid, the error

identifies its line in the CSV. The application calls the adapter through GameCatalogSource.

Strategy — GameRecommender.recommend and ByDurationStrategy.getComparator

```
public List<BoardGame> recommend(List<BoardGame> games, GameNight gameNight) {
    return strategy.recommend(games, gameNight);
}

protected Comparator<BoardGame> getComparator() {
    return Comparator.comparingInt(BoardGame::getDurationMinutes)
            .thenComparingInt(BoardGame::getDifficulty)
            .thenComparing(BoardGame::getTitle);
}
```

GameRecommender calls the selected strategy. The strategies share the filter for player count and available time. ByDurationStrategy puts the shortest games first. ByDifficultyStrategy puts the easiest games first.

Observer — ParticipantService.notifyObservers and RecommendationUpdater.onParticipantChanged

```
private void notifyObservers(ParticipantEvent event) {
    for (ParticipantObserver observer : List.copyOf(observers)) {
        observer.onParticipantChanged(event);
    }
}

public void onParticipantChanged(ParticipantEvent event) {
```

```
        refresh(event.getGameNight());
    }
```

MeetingController subscribes RecommendationUpdater to ParticipantService. When the service adds or removes a participant, it sends a ParticipantEvent to its observers. RecommendationUpdater gets the meeting from that event and runs the active recommendation strategy again.